MARK SCHEME · TEACHER COPY

10BR · Chapter 4: Software

Cambridge IGCSE Computer Science 0478

Paper 1 Computer Systems

Total marks: 39

Official Cambridge mark scheme extracts for each question in the matching exam paper, with question numbers changed to match the exam. Source of each question is shown above its rows.

For teacher / admin use only. Do not give to students before they have attempted the paper.

Mark scheme extracts © Cambridge University Press & Assessment.

Total: 39 marks

Question 1 · May/June 2024 Paper 12, original Q4 (a),(b),(c) [5 marks]

Question	Answer	Marks
1(a)	Any **one** from: • Operating system • Utility software // by example	1
1(b)	Any **two** from.: Example: • Word processor • Spreadsheet • Database • Web browser • Image/photo-editor • Video-editor • Email client • Game • Social media app	2
1(c)	Any **two** from: • System software **manages/maintains** the **hardware/software** • Applications software allows the **user** to perform tasks	2

Question 2 · May/June 2025 Paper 12, original Q3 (a),(b) [11 marks]

Question	Answer	Marks
2(a)	Any **two** from: • It is easier to debug. • Less likely to make errors. • The program is machine independent/portable	2

Total: 39 marks

2(b)(i)	**One** mark for each correct term in the correct place. • whole code • executing • all • line by line • error A compiler translates the **whole code** at once before **executing** it. A compiler produces an error report that displays **all** errors. An interpreter translates and executes the code **line by line**. An interpreter stops execution when an **error** is found and continues once it is corrected.	**5**
2(b)(ii)	**One** mark for each correct function. **One** mark for each correct matching role description. Examples: • Code editor • Allows the programmer to write/change the program. • Run-time environment • Allows the user to run the code and see the output. • Error **diagnostics** • Features that can be used to find errors in the code • Auto-completion • A programmer starts to type a command word, and the IDE suggests **option** for completing it. • Auto-correction • If a programmer **misspells** a command word it is changed to the correct spelling • Prettyprint • The **command words**/**identifiers** are given different colours	**4**

Question 3 · May/June 2026 Paper 11, original Q4 (d) [4 marks]

Question	Answer	Marks	Guidance
3 ·	Examiners must **consistently** apply the requirements of the mark scheme and any rules for what to do when candidates have not followed instructions correctly.		

Total: 39 marks

<table>
<tr><td>3(a)</td><td>One mark for each correct function name or role

<table><tr><th>function</th><th>role</th></tr><tr><td>providing an interface</td><td>• To allow a user to communicate/interact with the computer/device/system</td></tr><tr><td>managing memory</td><td>It makes sure that two processes do not access the same memory location at the same time.</td></tr><tr><td>managing interrupts</td><td>• Checks the source of the interrupt
• Assigns/checks interrupt priority // Finds highest priority interrupt // Decides which interrupts to action
• Controls/checks the interrupt queue
• Calls/uses the interrupt service routine/ISR</td></tr><tr><td>managing (user) accounts // providing/managing (system) security</td><td>It allows the user to set a username and password.</td></tr></table></td><td>4</td><td>Providing an interface – BOD hardware/software in place of computer

Providing an interface – if an example alone is given e.g. ‘It allows the use of buttons’, this is NE.

Managing interrupts – BOD reference to handling the queue in any correct way

Managing interrupts – allow any correct reference to dealing with the priority of an interrupt

Managing interrupts – allow reference to moving current CPU task to stack. The OS does not strictly do this, but it is a fundamental part of the managing interrupts process

Managing accounts – ‘manages users’ is TV</td></tr>
</table>

Question 4 · May/June 2026 Paper 11, original Q5 (a),(b),(c),(d) [10 marks]

Question	Answer	Marks	Guidance
4	Examiners must use the **full range of marks** defined in the mark scheme, unless limited by the quality of the candidate responses seen.		
4(a)	To translate/convert high-level language into low-level language/machine code/binary	**1**	Just translates code/program to low-level/machine code is NE
4(b)	Assembler	**1**	Mark the first answer given. Do not accept ‘assembly’.

4(c)	**One** mark for each correct letter **One** mark for each corrected statement • A • An interpreter translates and executes the code **one line/statement at a time** • E • A compiler produces an **executable** file	**4**	If an incorrect letter is given, do not read on to statement correction. Accept minor variations for correct statements that have the same meaning. For letter A we are looking for a correction on the use of variable. For letter E we are looking for a correction on the use of non-executable file. If no answer is given in answer space, look to see if indication is given in the table.

Total: 39 marks

4(d)	**One** mark for each correct function **One** mark for a matching purpose • Code editor • Allows the **user/programmer** to write/change the program • Run-time environment • Allows the user to **run/execute** the code and see the **output/result** // Allows the user to **input data** to **run/execute** the code // To allow the user to test the code • Error diagnostics // debugging tools // by example e.g. break points, stepping, variable watch window • Finds/identifies/detects errors in the code // by example • Auto-completion • Gives **options** to complete the remainder of a command word/key term/identifier/syntax (to limit errors/speed up development) // It **predicts** the rest of a command word/key term/identifier (to limit errors/speed up development) // It **predicts** the required syntax e.g. when a bracket is needed (to limit errors/speed up development) • Auto-correction • Corrects **spelling** errors in command words/key terms/identifiers/syntax (to limit errors in the code) • Prettyprint // syntax highlighting // colour coding • Changes the colour of text in the code to highlight command words/key terms/identifiers (to make them easier to identify) • Line numbers • To give (incremented) numbers to each line in the code (to allow a line in the code to be easily/quickly found)	**4**	Do not accept translator/interpreter/compiler, this is given in the question. Mark first function given. If the function name is TV/blank, read on to the purpose to see if a correct function name is included. Award both correct name and purpose if seen. Award the purpose if the function name is still TV but it is obvious what is meant. Code editor – BOD just editor alone Code editor purpose – BOD edit in place of write/change Error diagnostics – just 'debugging' alone is TV Just 'words' alone is TV for command word/key term/identifier/syntax Allow minor spelling variations on function names. It you see something that you think is a viable alternate to the function name, please refer to your TL. Other viable functions outside the syllabus may be seen, such as: • artificial intelligence • auto-documentation • expand/collapse code blocks • built-in libraries/functions In this case, accept the function and a correct description of the purpose. The description must be specific and not generic. Refer to TL if in doubt.

Question 5 · May/June 2026 Paper 12, original Q7 (a),(b),(c),(d),(e) [9 marks]

Question	Answer	Marks	Guidance

Total: 39 marks

5(a)	A language that is closer to the language processed/used by a computer
	// A language that is closer to binary/machine code
	// A language has that little abstraction from that processed by a computer
		1	Do not accept 'understood' in place of processed/used.
			
			'it uses/is binary' is TV for it is closer to binary.
			
			Do not accept that it is a language that uses mnemonics. This is not answering this question as not all do.
			
			Do not accept benefits e.g. it can directly manipulate the hardware. This is not answering the question.
			
			Do not accept marks for saying what it is not e.g. It is not close to human language.

5(b)	**Two** from: • Assembly (language) • Machine code // Binary • Object code	**2**	Mark first answer given. MP2 – BOD machine language
5(c)	Assembler	**1**	Mark first answer given.
5(d)	**Two** from: • The program doesn't need to be portable • Can **directly manipulate/access** the hardware // Can have low-level **control** of the device • The program will be faster to **execute/run** • The program can be written for **specific** hardware // Can use/access **specialised** hardware • The program uses **memory** more **efficiently** // The program is less memory intensive • The program would take up less **storage** space (in the vending machine) // The **file size** would be smaller (than the high-level equivalent file)	**2**	Mark whole answer. MP2 – program can communicate with hardware is TV as high-level language can also do this. There is no reference to it being direct. MP3 – Less time to translate is NE for execute MP5 – uses less space in memory is TV
5(e)	• Auto-correction • Run-time environment • PrettyPrint	**3**	Mark first answer given.